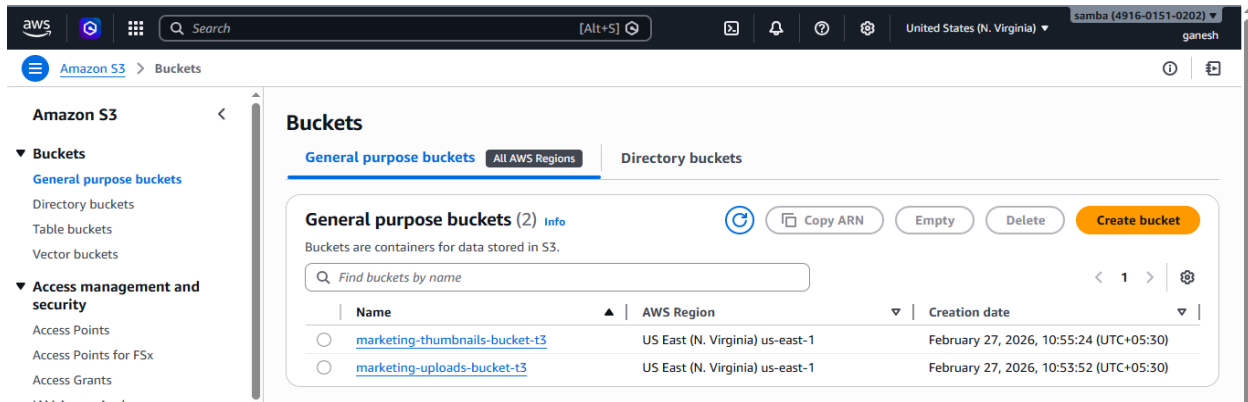


Serverless Image Resizer & Logger with S3 + Lambda

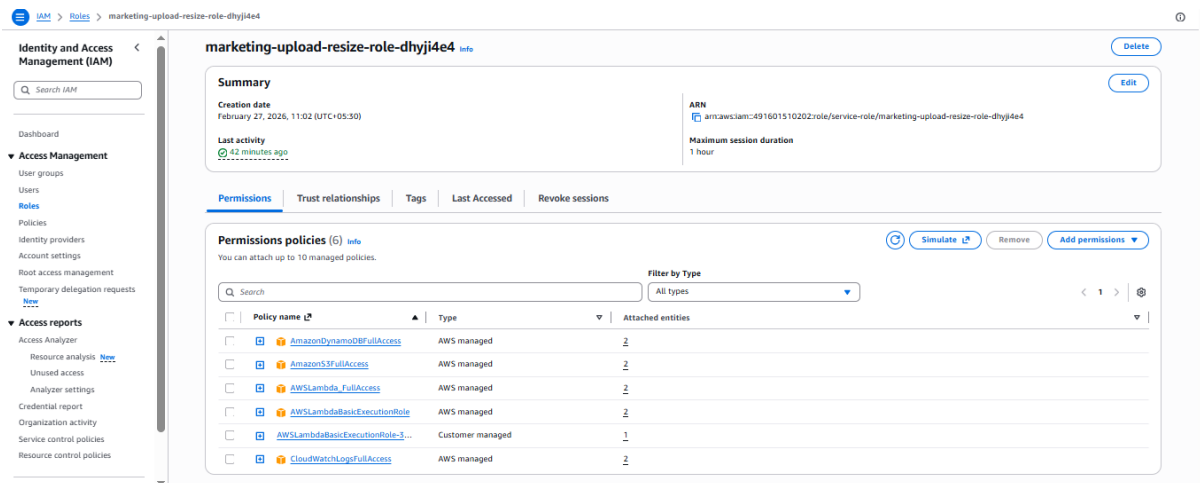
Step-1

1. Open the **Amazon S3 console** and select the **General purpose** buckets page.
2. Select the AWS Region closest to geographical location. must create Lambda function in the same Region.
3. First create two Amazon **S3 buckets**. The first bucket is the **source bucket** you will upload the images to. The second bucket is used by Lambda to save the **resized thumbnail**.



Step-2- Create an execution role

1. To create an execution role and attach your permissions policy (console)
2. Open the **Roles** page of the (IAM) console and Choose **Create role**.
3. For Trusted entity type, select AWS service, and for Use case, select Lambda.
4. Add the permissions policies and Choose Create role.



Step-3

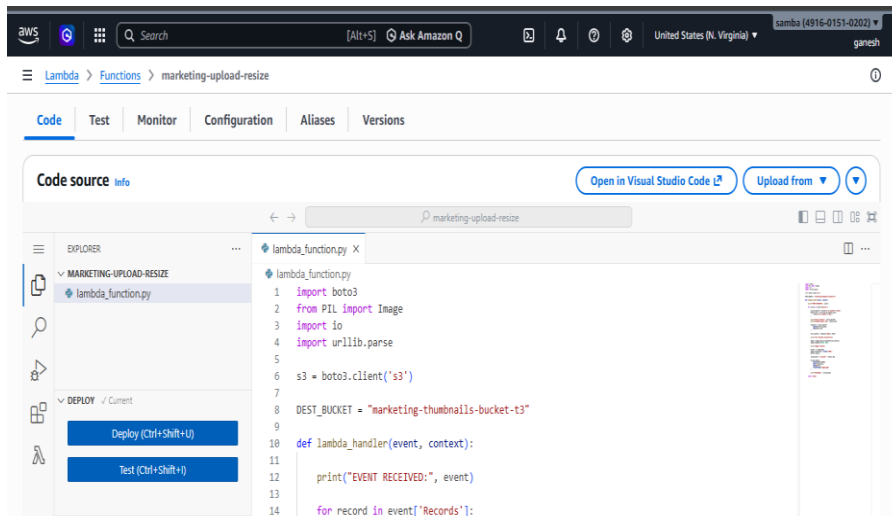
Create a Lambda function in the console using the Python 3.11 runtime.

1. Choose **Create function**.
2. Choose **Author from scratch**
3. Under **Basic information**, do the following:
 1. For **Function name**, **marketing-upload-resize**
 2. For **Runtime**, choose **Python 3.11**.
 3. For **Architecture**, choose **x86_64**.
4. In the **Change default execution role** tab, then Expand the tab, then choose **Create default role**.
5. Choose **Create function**.

The screenshot shows the 'Create function' page in the AWS Lambda console. At the top, there are three tabs: 'Author from scratch' (selected), 'Use a blueprint', and 'Container image'. Below the tabs is the 'Basic information' section. It includes a 'Function name' field with the value 'marketing-upload-resize', a 'Runtime' dropdown menu set to 'Python 3.11', and an 'Architecture' dropdown menu set to 'x86_64'. There is also a 'Permissions' section with a 'Change default execution role' button. Below this, the 'Execution role' section is expanded, showing 'Create default role' as the selected option. At the bottom right, there are 'Cancel' and 'Create function' buttons.

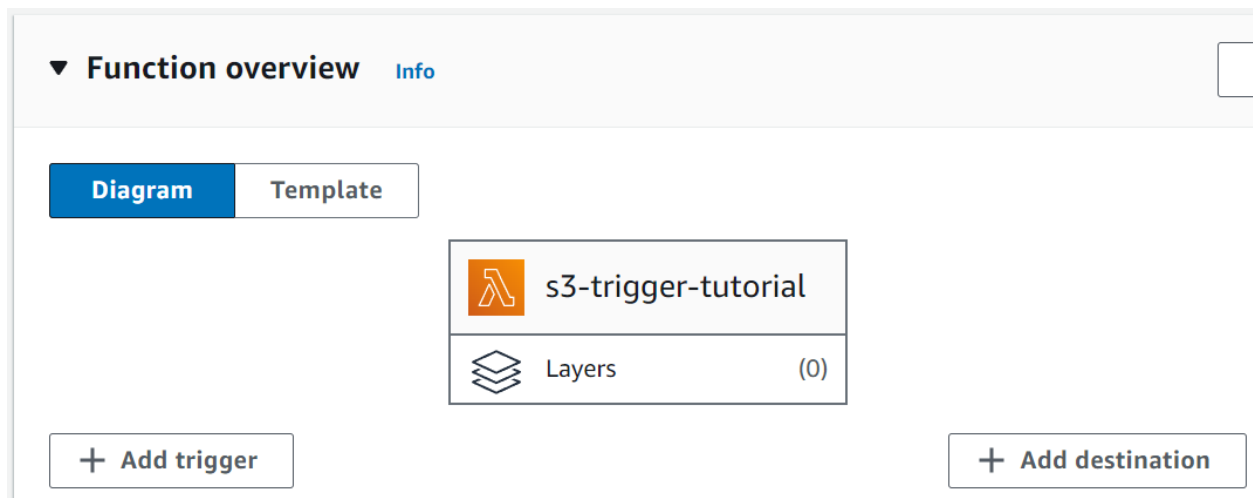
Step-4

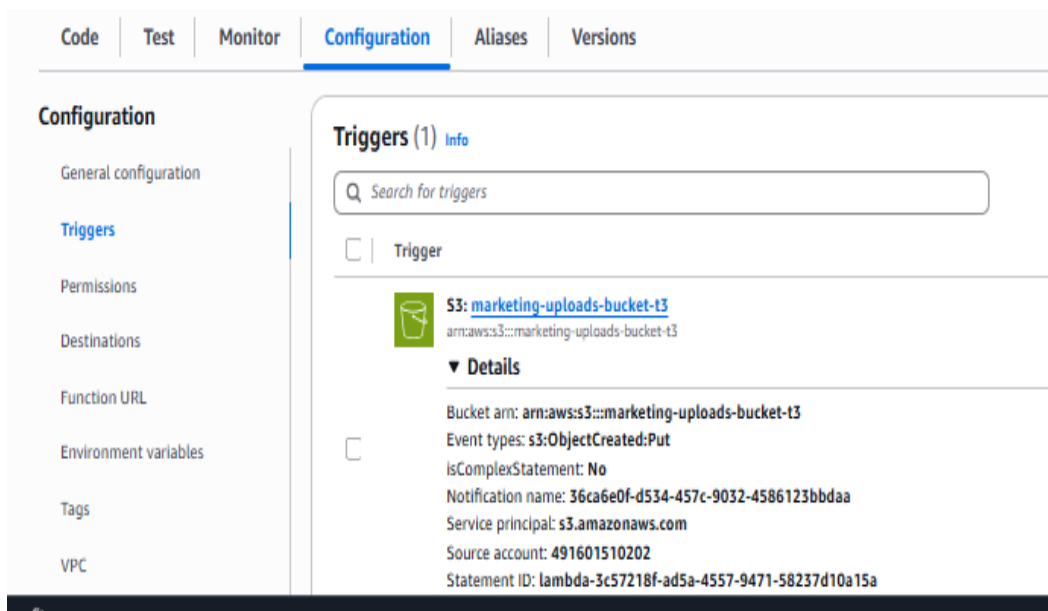
1. In the **Code source** pane on the Lambda console, paste the code into the code editor, replacing the code that Lambda created.
2. In the **DEPLOY** section, choose **Deploy** to update your function's code:



Step-4 -To create the Amazon S3 trigger

1. In the **Function overview** pane, choose **Add trigger**. Trigger configuration Select **S3**.
2. Under **Bucket**, select the source bucket created by earlier
3. Under **Event types**, be sure that **Put** is selected.
4. Under **Recursive invocation**, select the check box to acknowledge that using the same Amazon S3 bucket for input and output is not recommended. Choose **Add**.

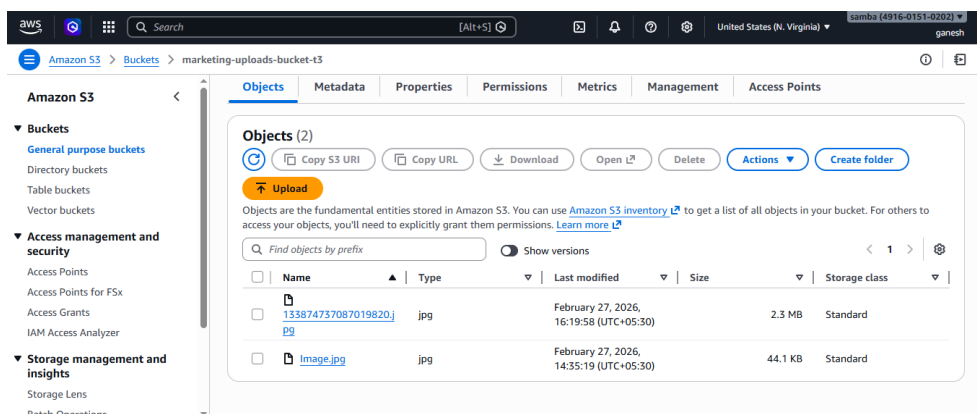




Step-5

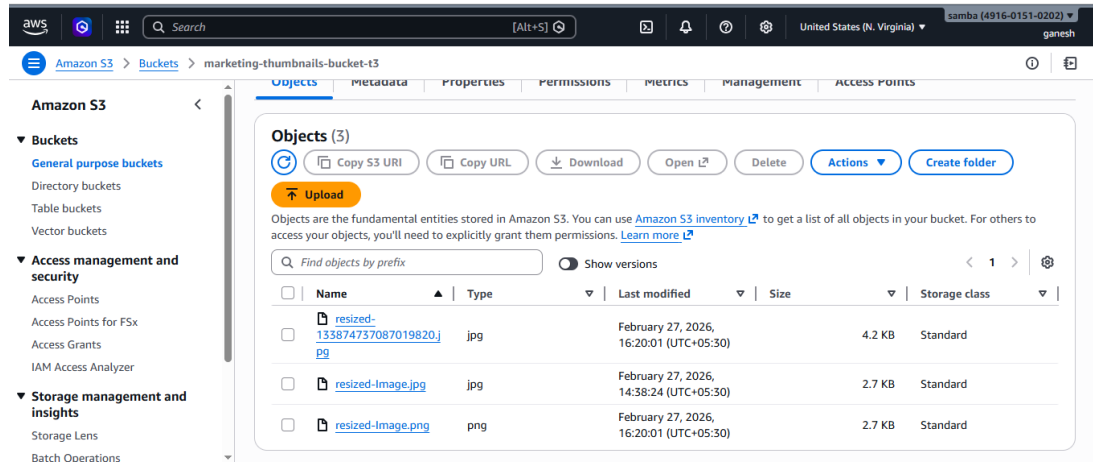
To test your Lambda function using the Amazon S3 trigger (console)

1. To upload an image to your Amazon S3 bucket, do the following:
 - a. Open the [Buckets](#) page of the Amazon S3 console and choose your source bucket.
 - b. Choose **Upload**.
 - c. Choose **Add files** and use the file selector to choose the image file you want to upload. Your image object can be any .jpg or .png file.
 - d. Choose **Open**, then choose **Upload**.



2. Verify that Lambda has saved a resized version of image file in target bucket by doing the following:

- Navigate back to the [Buckets](#) page of the Amazon S3 console and choose destination bucket.
- In the **Objects** pane, now see two resized image files, one from each test of Lambda function. To download your resized image, select the file, then choose **Download**.



Step-6

S3 log event is a message generated by Amazon S3 when something happens in a bucket (like file upload or delete). It does **NOT directly tell CloudWatch Logs**.

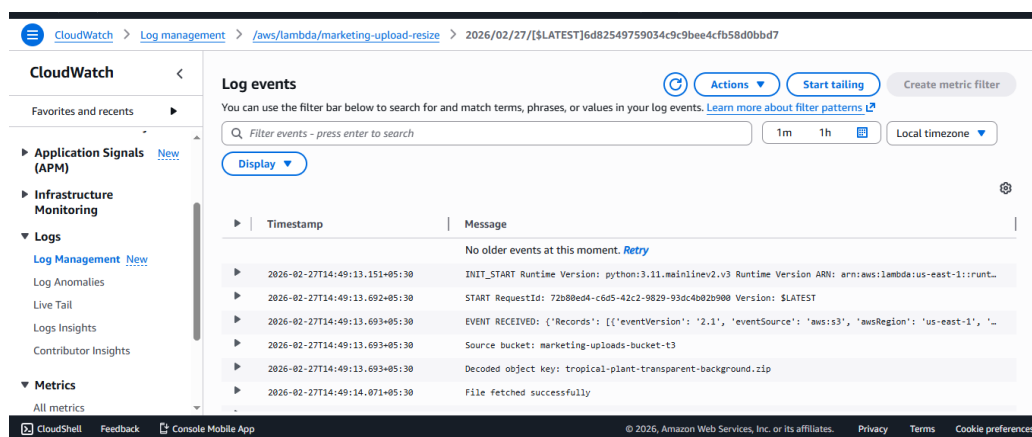
Instead:

- File Upload → S3 creates event → (Optional) Triggers Lambda → Lambda writes logs → CloudWatch Logs stores logs

Important Difference

- **S3 Event** → Created by S3 when an action happens
- **CloudWatch Logs** → Stores logs written by services like Lambda

S3 itself does not automatically send logs to CloudWatch Logs.



1. **Why use Lambda for image processing?**

AWS Lambda is ideal for image processing because it is serverless, automatically scalable, and cost-effective. When an image is uploaded to Amazon S3, Lambda can instantly resize, compress, or convert it without managing any servers. You only pay for the execution time used, making it efficient for workloads that run on demand. It also integrates seamlessly with S3 events, enabling real-time processing.

2. **What happens if multiple images are uploaded simultaneously?**

Lambda automatically scales. If multiple images are uploaded at the same time, AWS creates multiple Lambda instances to process them in parallel, depending on the concurrency limits.

3. **How does S3 event trigger Lambda?**

When a file is uploaded, S3 generates an event notification (like ObjectCreated:Put) and sends it to the configured Lambda function, which then starts execution automatically.

Conclusion

The Serverless Image Resizer solution uses S3 and Lambda to automatically resize images when they are uploaded. S3 triggers Lambda, which creates a thumbnail, stores it in another bucket, and logs details in CloudWatch.

This architecture is scalable, cost-effective, and requires no server management. It improves workflow efficiency while following secure and event-driven best practices.